

Episode 7: Detecting and Fixing Config Drift

Video Title

"Network as Code: When Someone SSHes In and Breaks Everything"

Target Length

18-22 minutes

Goal

Demonstrate the drift detection engine catching an out-of-band change. Show the three reconciliation strategies. Make the case that drift is the biggest threat to NaC adoption, and this lab solves it.

Pre-Recording Checklist

Before you hit record, make sure:

- ContainerLab topology is running with clean, deployed configs
 - All post-change tests passing (clean baseline)
 - Know exactly which manual change you will make for the demo
 - Drift detection script is working
 - Terminal ready with drift and reconciliation commands
-

SEGMENT 1: The Drift Problem (0:00 - 3:00)

What to show: Nothing on screen. This is a talking-to-camera segment.

Talking points:

- "You have a data model, validation, config generation, a CI/CD pipeline, and automated tests. The entire stack from Labs 1 through 5. It works beautifully, right up until someone SSHes into a router and types a command."
 - "That is configuration drift. The network no longer matches the source of truth. And it happens in every organization, no matter how disciplined the team is."
 - "Emergency fixes at 3 AM. A vendor TAC engineer who needs to test something. A junior engineer who does not know the pipeline exists yet. Someone who thinks 'it is just one command, I will update the YAML later' and then forgets."
 - "The Cisco NaC paper is clear: out-of-band changes are the enemy. The next pipeline run will see a difference between intent and reality. If you do not detect and manage drift, the pipeline becomes untrustworthy."
 - "Lab 6 builds a drift detection engine that catches these changes and gives you three ways to handle them."
-

SEGMENT 2: How Drift Detection Works (3:00 - 5:00)

What to show: Simple diagram or just talking points.

Talking points:

- "The concept is simple. The data model says what the network should look like. The devices have their running configs. The drift engine compares the two."
 - "It connects to every device via Scrapli, collects the running config, parses it into structured data, and compares it against what the data model says the config should be."
 - "If there is a difference, that is drift. The engine produces a report: which device, which section, what changed, and when it was detected."
 - "This runs on a schedule. Every hour, every 15 minutes, whatever your organization needs. A GitHub Actions cron job triggers it automatically."
-

SEGMENT 3: Live Demo - Creating Drift (5:00 - 8:00)

What to show: Make a manual change on a device.

Step 1: Verify clean state

```
uv run python -m drift.detect
```

- "Clean. No drift detected. The network matches the data model perfectly."

Step 2: Make an out-of-band change

```
docker exec clab-nac-spine-leaf-leaf1 vtysh -c "configure terminal" -c "interface lo100" -c "ip address 10.99.99.1/32"
```

- "I just SSHed into leaf1 and added a loopback interface with an IP address. This change did not go through the pipeline. It is not in the data model. It is not in Git. Nobody reviewed it."

Step 3: Run drift detection

```
uv run python -m drift.detect
```

Talking points:

- "Drift detected on leaf1. The engine found an interface and IP address that exist on the device but not in the data model."
 - "Look at the report. It tells you exactly what changed: interface lo100 with IP 10.99.99.1/32 was added. It shows the diff between intended and actual."
 - "If this were running on a schedule, your team would get an alert within minutes. Instead of discovering this change weeks later when something breaks, you know about it immediately."
-

SEGMENT 4: Reconciliation Strategy 1 - Auto-Remediate (8:00 - 11:00)

What to show: Run the auto-remediation.

```
uv run python -m drift.reconcile --strategy remediate
```

Talking points:

- "Strategy one: auto-remediate. The engine generates a config that removes the drift and pushes it to the device. The network returns to the intended state."
- "This is the right strategy for low-risk drift. Someone added an interface description. Someone changed a timer. Things that are clearly not intentional and safe to revert."

Verify:

```
uv run python -m drift.detect
```

- "Clean again. The unauthorized change is gone. The network matches the data model."

Now recreate the drift for the next demo:

```
docker exec clab-nac-spine-leaf-leaf1 vtysh -c "configure terminal" -c "interface lo100" -c "ip address 10.99.99.1/32"
```

SEGMENT 5: Reconciliation Strategy 2 - PR-Based Review (11:00 - 14:00)

What to show: Run the PR-based reconciliation.

```
uv run python -m drift.reconcile --strategy review
```

Talking points:

- "Strategy two: create a PR. Instead of auto-remediating, the engine generates a remediation config and opens a pull request for human review."
- "This is the right strategy for anything that could affect traffic. Someone changed a BGP timer. Someone modified a route policy. You want a human to look at it before reverting."
- "The PR shows exactly what will change and why. The reviewer can approve the revert or investigate further."
- "This is also the strategy for when you are not sure if the change was intentional. Maybe it was an emergency fix. Maybe it needs to stay. The PR creates a conversation."

SEGMENT 6: Reconciliation Strategy 3 - Absorb (14:00 - 17:00)

What to show: Run the absorb strategy.

```
uv run python -m drift.reconcile --strategy absorb
```

Talking points:

- "Strategy three: absorb the change. This is the opposite of remediation. Instead of reverting the device to match the data model, you update the data model to match the device."
- "When would you use this? Emergency fixes. Something was broken in production, an engineer SSHed in and fixed it. The fix is correct. It needs to stay. But it also needs to be in the data model so the next pipeline run does not revert it."

- "The absorb strategy captures the out-of-band change and adds it to the YAML files. It then opens a PR so the team can review and merge the updated data model."
- "This is the bridge between 'we do Network as Code' and 'sometimes we have to do things manually.' It acknowledges reality without abandoning the process."

Verify:

```
uv run python -m drift.detect
```

- "Clean. But this time, the data model changed to match the device, not the other way around."
-

SEGMENT 7: Scheduled Detection (17:00 - 19:00)

What to show: Reference the `drift_check.yaml` workflow from Lab 4.

Talking points:

- "In production, drift detection runs on a schedule. The `drift_check.yaml` workflow in GitHub Actions triggers on a cron schedule."
 - "Every hour, every 30 minutes, whatever makes sense for your environment. It connects to all devices, checks for drift, and alerts the team if anything is found."
 - "The alert includes the drift report: which devices, what changed, and a recommended reconciliation strategy based on the severity of the drift."
 - "This closes the loop. The pipeline deploys changes. The tests verify they worked. And the drift engine watches for unauthorized changes between deployments."
-

SEGMENT 8: The Close (19:00 - 21:00)

What to show: Series overview or back to camera.

Talking points:

- "That is Lab 6. Drift detection that catches out-of-band changes, and three reconciliation strategies: auto-remediate, PR-based review, and absorb."
 - "This is the lab that makes Network as Code practical for real organizations. You cannot tell a team 'never SSH into a device again.' But you can detect when it happens and manage the aftermath."
 - "The build guide walks through building the drift engine, writing the comparison logic, and setting up the scheduled pipeline. Available at [your link] for \$49."
 - "Next episode is the finale. We connect an AI agent to the entire stack. It answers questions about the fabric using live data, runs on local open-source models, and ties together everything from Labs 1 through 6. That is Lab 7."
 - "Subscribe, and I will see you in the lab."
-

Commands Reference (Quick Copy)

```
# Drift detection
```

```
uv run python -m drift.detect

# Reconciliation strategies
uv run python -m drift.reconcile --strategy remediate
uv run python -m drift.reconcile --strategy review
uv run python -m drift.reconcile --strategy absorb

# Create manual drift for demo
docker exec clab-nac-spine-leaf-leaf1 vtysh -c "configure terminal" -c "interface lo100" -c "ip address 10.99.99.1/32"

# Remove manual drift
docker exec clab-nac-spine-leaf-leaf1 vtysh -c "configure terminal" -c "no interface lo100"
```

Do NOT Show On Camera

- The drift detection script internals (paid content)
- The reconciliation logic (paid content)
- The comparison/diffing code (paid content)
- The brownfield import tool (paid content)
- Step-by-step build process (paid content)

DO Show On Camera

- Drift detection output (clean and with drift)
- The manual change being made on the device
- Drift reports showing what changed
- Reconciliation output for all three strategies
- The concept of scheduled detection
- Before/after state of the network